

Chapter 47. Data Quality, Observability & Contracts

Testing data in the pipeline, write-audit-publish, freshness and volume monitoring, lineage, data contracts, incidents, and alerts people still read.

Draft chapter

Contents

Chapter 47. Data Quality, Observability & Contracts

Why this matters

In plain English

47.1 The dimensions of data quality

47.2 Testing data inside the pipeline

47.3 A check library for Riverstone

47.4 Write-audit-publish

47.5 Observability: freshness, volume, and unusual values

47.6 Lineage: what breaks when this breaks

47.7 Data contracts in practice

47.8 Handling a data incident

47.9 Alerts people still read

47.10 The tools landscape

Common mistakes and how to spot them

In the real world: the dashboard that was right but late

Tools

The project: quality, monitoring, and a contract for the Riverstone pipeline

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 47. Data Quality, Observability & Contracts

Part V — Data Engineering, Integration & Scale

Chapter at a glance

You will learn to: name the dimensions of data quality and write a test for each · build a small test framework where every test is a query that returns the rows that break a rule · choose severities so that only real problems stop a pipeline · use write-audit-publish so bad data never reaches the tables people read · monitor freshness and volume, and spot unusual days against history · trace lineage to see which reports a broken table affects · write a data contract with a source owner and check it automatically · run a data incident: severity, communication, fix, and review · keep alerts few enough that people still read them · compare dbt tests, Great Expectations, Soda, and observability platforms.

Before you start: Chapter 45 (ingestion and reconciliation), Chapter 46 (the orchestrated pipeline), and Chapter 14 (finding and fixing data problems as an analyst).

Time needed: 12–16 hours of reading and practice, spread over two to three weeks.

Tools: Python 3.12 with `duckdb` and `psycopg2`, PostgreSQL 16, and the Chapter 47 companion folder. Everything runs on your own computer. No data quality product is needed: the tests are SQL.

Practice data: the Chapter 45 and 46 environment: `riverstone_source` (the ERP and CRM), the dispatch files, and the warehouse you've been building.

Why this matters

Chapter 1 told you that analysts check data before they trust it, and promised that Chapter 47 would show how data teams do that automatically. Chapter 13 built the habit of reconciling every number against a source. Chapter 46 added one check to a pipeline and used it to stop a wrong report from being sent.

This chapter turns those habits into a system, because as soon as more than one person depends on the data, checking by hand stops working. Somebody is on leave. Somebody assumes somebody else looked. And the failures that matter most are the quiet ones: a load that brings in nothing, a status nobody agreed to, a file that's three days old, a column renamed by a helpful colleague.

There's also a gap left over from Chapter 46. When the check failed on 5 January, the report wasn't sent, which was right. But the wrong row was still sitting in the table, visible to anyone who queried it. This chapter closes that gap, and adds the monitoring, lineage, contracts, and incident habits that make a data platform something people trust.

In plain English

Think about a factory's quality department, because Riverstone has one.

- **Specifications** say what "good" means: this crate weighs between 4.8 and 5.2 kg, this lid fits this base. Data has specifications too, and writing them down is most of the work.
- **Checks on the line** catch problems while the product is being made, not after it ships. That's **testing data inside the pipeline**.
- **Quarantine** holds a doubtful batch until someone inspects it. Nothing doubtful goes to customers. That's **write-audit-publish**.
- **A goods-inward check** confirms what arrived from a supplier matches what was agreed. That's a **data contract**.
- **A gauge that hasn't moved in three days** is more worrying than a gauge reading badly, because nobody noticed. That's **freshness monitoring**.
- **Batch tracing** tells you which customers got product from a faulty batch. That's **lineage**.
- **An alarm that rings all day** gets switched off, and then nobody hears the real one. That's **alert fatigue**.

A data team is the quality department for the numbers. This chapter is its handbook.

47.1 The dimensions of data quality

"Quality" is too vague to test. Break it into dimensions, and each one becomes a rule you can write in SQL.

Dimension	The question it answers	Riverstone example	Typical test
Accuracy	Do the numbers match reality, or a trusted source?	Warehouse revenue for a day equals the ERP's	Reconciliation against the source
Completeness	Is anything missing?	Every order has at least one order line; every order has a customer	Missing values; counts against the source
Validity	Does each value follow its rules?	<code>status</code> is one of Pending, Shipped, Delivered, Cancelled; quantities are positive	Accepted values; ranges
Uniqueness	Is anything counted twice?	One row per order in <code>raw.orders</code> ; one Flash row per day	Duplicate key tests
Consistency	Do related things agree?	Every order line points at an order that exists; dispatch rows point at known orders	Relationship tests across tables
Timeliness	Is it here, and recent enough?	Yesterday's orders are loaded by 6:30 a.m.; the dispatch file arrived last night	Freshness checks

Two more properties matter and aren't tested the same way. **Usability** is whether people can find and understand the data: names, definitions, and documentation (Chapter 32 and Chapter 64). **Trust** is whether they believe it, which is earned by the other dimensions holding for months.

Simplification note. Different books and tools name these slightly differently, and some add dimensions such as integrity or conformity. The names matter less than the habit: for every important table, write down what "good" means, then test it.

47.2 Testing data inside the pipeline

A **data test** is a rule about data, expressed so a computer can judge it. The most useful way to write one, and the way most tools work underneath, is this:

Write a query that returns the rows that break the rule. If it returns no rows, the test passes.

That single idea covers almost everything: duplicates, missing values, invalid values, broken relationships, and totals that don't match a source. It also gives you something valuable when a test fails: the offending rows, which is where investigation starts.

Severity: stop, or warn?

Not every problem should stop a pipeline at 6:30 a.m.

- **Error:** the data is wrong in a way that would mislead someone. Stop, and don't publish. Example: the warehouse's revenue doesn't match the ERP.
- **Warning:** something is odd and worth a look, but publishing is still better than not publishing. Example: one dispatch row points at an order the warehouse doesn't have yet.

Start strict on the numbers people act on, and lenient on the edges. A test whose failures are routinely ignored should either be fixed, downgraded to a warning, or deleted. Tests nobody acts on are worse than no tests, because they train people to ignore red.

Where checks belong

Layer	What to test	Why here
Raw	Structure and load completeness: expected columns, row counts against the source, keys unique	Catches ingestion problems before anything is built on them
Staging	Validity and consistency: types converted, accepted values, relationships between tables	Catches source data problems and conversion mistakes
Mart	Business rules and reconciliation: one row per day, totals match the source, no negative revenue	Catches modeling mistakes, and protects the numbers people read
Before delivery	The blocking checks from Chapter 46	Nothing wrong leaves the platform

47.3 A check library for Riverstone

Setting up

The companion folder `companion/ch47/` is the Chapter 46 environment with a new reset script. Start Python in that folder:

```
import os, json
from datetime import datetime, timedelta
import duckdb, dagster as dg
from reset_ch47 import reset
from apply_day import apply_day
import ingest

reset()
wh = duckdb.connect("warehouse/riverstone_wh.duckdb")
apply_day(1) # the 2 January business day, from Chapter 46
for table in ["orders", "order_items"]:
    ingest.sync_table(wh, table)
ingest.load_dispatch(wh, "2026-01-02")
print("rows loaded:", wh.execute("SELECT (SELECT COUNT(*) FROM raw.orders), (SELECT COUNT(*) FROM raw.order_items), (SELECT COUNT(*) FROM raw.dispatch)").fetchone())
```

```
rows loaded: (177, 333, 5)
```

The warehouse now holds the 2 January state from Chapter 46: 177 orders, 333 order lines, and 5 dispatch rows.

A tiny test framework

Every test is a query. The runner runs them and reports.

```
def data_test(name, sql, severity="error", description=""):
    """A data test is a query that returns the rows that BREAK a rule.
    Zero rows means the test passed."""
    return {"name": name, "sql": sql, "severity": severity, "description": description}

def run_tests(tests, show_rows=2):
    results = []
    for t in tests:
        rows = wh.execute(t["sql"]).fetchall()
        results.append({"name": t["name"], "severity": t["severity"], "failing_rows": len(rows),
            "sample": rows[:show_rows]})
    width = max(len(r["name"]) for r in results)
    for r in results:
        status = "PASS" if r["failing_rows"] == 0 else r["severity"].upper()
        print(f"{r['name']:<{width}} {status:<5} failing rows: {r['failing_rows']}"
            + (f" e.g. {r['sample']}" if r["failing_rows"] else ""))
    errors = [r for r in results if r["failing_rows"] and r["severity"] == "error"]
    print(f"\n{len(results)} tests, {sum(1 for r in results if not r['failing_rows'])} passed, "
        f"{len(errors)} failed as errors, {sum(1 for r in results if r['failing_rows'] and r['severity'] == 'warn')} warnings")
    return results

print("test framework ready")
```

How it works, line by line.

- `data_test()` records a test's name, the query that returns *failing* rows, its severity, and a plain-English description of the rule. Writing the description first is a good habit: if you can't say the rule in one sentence, you don't have a rule yet.
- `run_tests()` runs each query, counts the rows it returns, and prints one line per test with a sample of the failures. It returns the results, so a pipeline can decide what to do.
- The last line separates **errors** from **warnings**, which is what section 47.4 uses to decide whether to publish.

This is a teaching version, in about 15 lines. dbt, Great Expectations, and Soda add scheduling, storage of results, history, and reporting, but underneath they run tests exactly like these.

The tests

```
TESTS = [
  data_test("orders_unique_key", ""
    SELECT order_id, COUNT(*) AS n FROM raw.orders GROUP BY order_id HAVING COUNT(*) > 1"",
    description="uniqueness: one row per order"),
  data_test("orders_customer_not_null", ""
    SELECT order_id FROM raw.orders WHERE customer_id IS NULL"",
    description="completeness: every order has a customer"),
  data_test("orders_status_accepted", ""
    SELECT order_id, status FROM raw.orders
    WHERE status NOT IN ('Pending', 'Shipped', 'Delivered', 'Cancelled')"",
    description="validity: only known statuses"),
  data_test("order_items_have_an_order", ""
    SELECT i.order_item_id, i.order_id FROM raw.order_items AS i
    LEFT JOIN raw.orders AS o ON o.order_id = i.order_id
    WHERE o.order_id IS NULL"",
    description="consistency: every order line belongs to an order"),
  data_test("order_items_quantity_positive", ""
    SELECT order_item_id, quantity FROM raw.order_items WHERE quantity <= 0"",
    description="validity: quantities are positive"),
  data_test("orders_have_at_least_one_line", ""
    SELECT o.order_id FROM raw.orders AS o
    LEFT JOIN raw.order_items AS i ON i.order_id = o.order_id
    WHERE o.order_date >= DATE '2026-01-01' AND i.order_item_id IS NULL"",
    description="completeness: orders this year have order lines"),
  data_test("dispatch_order_known", ""
    SELECT d.dispatch_id FROM raw.dispatch AS d
    LEFT JOIN raw.orders AS o ON o.order_id = d.order_id
    WHERE o.order_id IS NULL"",
    severity="warn",
    description="consistency: dispatches point at known orders"),
]
results = run_tests(TESTS)
```

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     PASS    failing rows: 0
order_items_have_an_order  PASS    failing rows: 0
order_items_quantity_positive PASS    failing rows: 0
orders_have_at_least_one_line PASS    failing rows: 0
dispatch_order_known       PASS    failing rows: 0
```

7 tests, 7 passed, 0 failed as errors, 0 warnings

Reading it. Seven rules, covering five of the six dimensions, and all passing on today's data. Notice `dispatch_order_known` is a **warning**: dispatch rows can legitimately arrive for an order the warehouse hasn't loaded yet, so it's worth seeing but shouldn't stop the Flash.

Notice also what these tests are *not*: they aren't a check that the pipeline ran. Chapter 46's checks answered "did this step produce the right numbers?"; these answer "is the data in the warehouse sane?" Both matter, and both run in the pipeline.

Watching them fail

A test suite nobody has seen fail is a test suite you don't know works. Break things on purpose.

```
try:
    conn = ingest.psycpg2.connect(ingest.SRC)
    with conn, conn.cursor() as cur:
        cur.execute("INSERT INTO order_items VALUES (335, 99999, 101, 12, 430.00, 0.00)")
    conn.close()
except Exception as e:
    print("the ERP refused it:", str(e).splitlines()[0])

# ... so break the warehouse instead, the way a half-finished load would
wh.execute("INSERT INTO raw.order_items VALUES (335, 99999, 101, 12, 430.00, 0.00)") # order that
# isn't there
wh.execute("INSERT INTO raw.order_items VALUES (336, 10176, 103, -8, 115.00, 0.00)") # negative
# quantity
wh.execute("UPDATE raw.orders SET status = 'On Hold' WHERE order_id = 10177") # status
# nobody agreed
print()
results = run_tests(TESTS)
```

the ERP refused it: insert or update on table "order_items" violates foreign key constraint
"order_items_order_id_fkey"

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     ERROR   failing rows: 1  e.g. [(10177, 'On Hold')]
order_items_have_an_order  ERROR   failing rows: 1  e.g. [(335, 99999)]
order_items_quantity_positive ERROR   failing rows: 1  e.g. [(336, -8)]
orders_have_at_least_one_line PASS    failing rows: 0
dispatch_order_known       PASS    failing rows: 0
```

7 tests, 4 passed, 3 failed as errors, 0 warnings

Reading it. Two lessons in one output.

First, the ERP **refused** the orphan order line: the source database has a **foreign key** (Chapter 12) preventing a line from pointing at an order that doesn't exist. Source systems often have constraints like this, which is why many warehouse problems come from the loading, not from the source.

So the second attempt broke the *warehouse* instead, which is exactly what a half-finished load, a bad manual fix, or a bug in a transformation does. Three tests caught three different problems: an unknown status, an order line with no order, and a negative quantity. Each failure shows the rows involved, so the investigation starts with facts.

Self-healing loads

Because Chapter 45's hash comparison makes the warehouse match the source (inserting, updating, *and* deleting), re-running the load repairs damage of this kind:

```
ingest.sync_table(wh, "orders")
ingest.sync_table(wh, "order_items")
results = run_tests(TESTS)
```

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     PASS    failing rows: 0
order_items_have_an_order  PASS    failing rows: 0
order_items_quantity_positive PASS    failing rows: 0
orders_have_at_least_one_line PASS    failing rows: 0
dispatch_order_known       PASS    failing rows: 0
```

```
7 tests, 7 passed, 0 failed as errors, 0 warnings
```

That's worth knowing during an incident: for tables loaded by full comparison, "re-run the load" is often the fix, and it's safe because the load is idempotent (Chapter 46).

The same tests as dbt tests

If your transformations live in dbt (Chapter 32), most of these tests are one or two lines of YAML, and dbt generates the SQL:

```
# models/schema.yml
version: 2
models:
  - name: stg_orders
    description: "One row per order, from the ERP"
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests: [not_null]
      - name: status
        tests:
          - accepted_values:
              values: ["Pending", "Shipped", "Delivered", "Cancelled"]
  - name: stg_order_items
    columns:
      - name: order_id
        tests:
          - relationships:
              to: ref('stg_orders')
              field: order_id
      - name: quantity
        tests:
          - dbt_utils.accepted_range:      # from the dbt-utils package
              min_value: 1
```

Custom rules (such as "orders this year have at least one line") are written as a SQL file that returns failing rows: the same idea as `data_test()`. Run them with `dbt test`, and fail the pipeline if any error-severity test fails.

47.4 Write-audit-publish

Chapter 46 left a gap. The check stopped the email, but the wrong row stayed in `mart.daily_flash`, where any dashboard or analyst could read it. **Write-audit-publish** (WAP) closes it:

1. **Write** the new data somewhere readers don't look (an audit or staging schema).
2. **Audit** it: run the tests against the new data.
3. **Publish** only if the tests pass: swap it into the table people read, in one transaction.

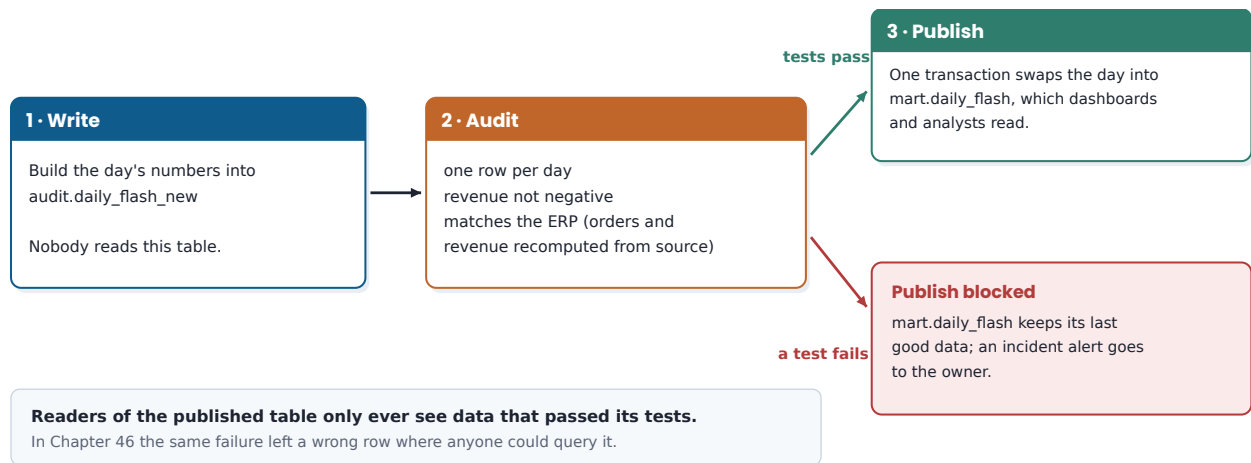


Figure 47.1 – Write-audit-publish. Readers of the published table only ever see data that passed its tests.

```

def build_flash_staging(day):
    wh.execute("CREATE SCHEMA IF NOT EXISTS mart")
    wh.execute("CREATE SCHEMA IF NOT EXISTS audit")
    wh.execute("""CREATE TABLE IF NOT EXISTS mart.daily_flash (
        flash_date DATE, orders_booked INTEGER, revenue_booked DECIMAL(12,2))""")
    wh.execute("CREATE OR REPLACE TABLE audit.daily_flash_new AS " + f"""
        SELECT DATE '{day}' AS flash_date,
            COUNT(DISTINCT o.order_id) AS orders_booked,
            COALESCE(SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)), 0) AS
revenue_booked
        FROM raw.orders AS o
        JOIN raw.order_items AS i ON i.order_id = o.order_id
        WHERE o.order_date = DATE '{day}' AND o.status <> 'Cancelled'""")

AUDIT_TESTS = [
    data_test("flash_one_row", "SELECT flash_date, COUNT(*) FROM audit.daily_flash_new GROUP BY 1
HAVING COUNT(*) <> 1"),
    data_test("flash_revenue_not_negative", "SELECT * FROM audit.daily_flash_new WHERE revenue_booked
< 0"),
    data_test("flash_matches_erp", """
        SELECT n.orders_booked AS warehouse_orders, e.orders_booked AS erp_orders,
            n.revenue_booked AS warehouse_revenue, e.revenue_booked AS erp_revenue
        FROM audit.daily_flash_new AS n, erp_expected AS e
        WHERE n.orders_booked <> e.orders_booked OR n.revenue_booked <> e.revenue_booked"""),
]

def publish_flash(day):
    src_n, src_rev = ingest.fetch_rows("""
        SELECT COUNT(DISTINCT o.order_id),
            COALESCE(SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)), 0)
        FROM orders AS o JOIN order_items AS i ON i.order_id = o.order_id
        WHERE o.order_date = %s AND o.status <> 'Cancelled'""", [day])[0]
    wh.execute("CREATE OR REPLACE TABLE erp_expected AS SELECT ? AS orders_booked, ?::DECIMAL(12,2)
AS revenue_booked", [src_n, src_rev])
    print(f"--- write-audit-publish for {day} ---")
    results = run_tests(AUDIT_TESTS)
    if any(r["failing_rows"] and r["severity"] == "error" for r in results):
        print("PUBLISH BLOCKED: mart.daily_flash left untouched")
        return False
    wh.execute("BEGIN")
    wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
    wh.execute("INSERT INTO mart.daily_flash SELECT * FROM audit.daily_flash_new")
    wh.execute("COMMIT")
    print("PUBLISHED")
    return True

build_flash_staging("2026-01-02")
publish_flash("2026-01-02")
print("mart.daily_flash:", wh.execute("SELECT * FROM mart.daily_flash ORDER BY
flash_date").fetchall())

```

```

--- write-audit-publish for 2026-01-02 ---
flash_one_row          PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp       PASS    failing rows: 0

3 tests, 3 passed, 0 failed as errors, 0 warnings
PUBLISHED
mart.daily_flash: [(datetime.date(2026, 1, 2), 2, Decimal('38710.00'))]

```

How it works. `build_flash_staging()` writes the day's numbers into `audit.daily_flash_new`. `publish_flash()` first computes the ERP's own answer and stores it as `erp_expected`, so the reconciliation test is a query like any other. It runs the audit tests; if any error-severity test fails, it stops and leaves the published table untouched. Otherwise it replaces the day's row in one transaction, exactly as in Chapter 46.

The same bad morning, with WAP

Now repeat Chapter 46's 5 January failure: the order-line load silently loads nothing.

```
apply_day(2)
ingest.BROKEN_ITEMS_SYNC = True          # the Chapter 46 bug: order lines load nothing
ingest.sync_table(wh, "orders"); ingest.sync_table(wh, "order_items")

build_flash_staging("2026-01-05")
publish_flash("2026-01-05")
print("mart.daily_flash:", wh.execute("SELECT * FROM mart.daily_flash ORDER BY
flash_date").fetchall())
print("readers of mart.daily_flash never saw the 5 January row:",
      wh.execute("SELECT COUNT(*) FROM mart.daily_flash WHERE flash_date = '2026-01-
05'").fetchone()[0] == 0)
```

```
--- write-audit-publish for 2026-01-05 ---
flash_one_row          PASS    failing rows: 0
flash_revenue_not_negative PASS    failing rows: 0
flash_matches_erp      ERROR    failing rows: 1  e.g. [(0, 1, 0.0, Decimal('18750.00'))]

3 tests, 2 passed, 1 failed as errors, 0 warnings
PUBLISH BLOCKED: mart.daily_flash left untouched
mart.daily_flash: [(datetime.date(2026, 1, 2), 2, Decimal('38710.00'))]
readers of mart.daily_flash never saw the 5 January row: True
```

Reading it. The reconciliation test failed: the ERP has 1 order for ₹18,750.00, the new data has 0 orders and ₹0.00. Publishing was blocked, and `mart.daily_flash` still holds only the good 2 January row. In Chapter 46, the same failure left a wrong row in the published table. Now readers never see it: they see yesterday's data, and the freshness check in the next section is what tells them (and the on-call engineer) that today's is missing.

Watch out: "no data" is also a state people can misread. A missing day can look like a zero day on a chart. Publish a status alongside the data (for example, a `load_status` table saying which days are published, late, or failed), and show it on dashboards, so "we don't know yet" never looks like "we sold nothing".

Variations

- **Blue/green tables:** build a whole new table, test it, then swap names. Good for full rebuilds.
- **Table snapshots or time travel:** warehouses such as Snowflake, BigQuery, and Delta or Iceberg tables (Chapter 49) can keep previous versions, so publishing is a metadata change and rolling back is quick.
- **Views over the last good partition:** readers query a view that points at published partitions only.

47.5 Observability: freshness, volume, and unusual values

Tests answer "is this data right?" **Observability** answers "is the platform behaving normally?" Three signals carry most of the value.

Freshness

Freshness is the age of the newest data. Stale data is the most common data incident, and the hardest to notice, because nothing failed: a report keeps showing last Tuesday's numbers.

```
NOW = datetime(2026, 1, 6, 6, 30)          # the pipeline's 6:30 a.m. run, as a fixed clock
for this example

def freshness_check(table, date_column, expected_within_hours, now=NOW):
    newest = wh.execute(f"SELECT MAX({date_column}) FROM {table}").fetchone()[0]
    newest_dt = datetime.combine(newest, datetime.min.time()) if not isinstance(newest, datetime)
    else newest
    age_hours = (now - newest_dt).total_seconds() / 3600
    status = "PASS" if age_hours <= expected_within_hours else "STALE"
    print(f"{table:<18} newest {date_column}: {newest} age: {age_hours:>6.1f} h limit:
{expected_within_hours} h {status}")
    return status == "PASS"

freshness_check("raw.orders", "order_date", 48)
freshness_check("raw.dispatch", "file_day", 48)
freshness_check("mart.daily_flash", "flash_date", 48)
```

```
raw.orders      newest order_date: 2026-01-05 age:  30.5 h limit: 48 h PASS
raw.dispatch    newest file_day: 2026-01-02 age: 102.5 h limit: 48 h STALE
mart.daily_flash newest flash_date: 2026-01-02 age: 102.5 h limit: 48 h STALE
```

Reading it. Orders are fresh: the newest is from 5 January, about 30 hours before this 6:30 a.m. run. Two things are stale: the dispatch file hasn't arrived since 2 January, and `mart.daily_flash` has no 5 January row because publishing was blocked. Both are real problems that no test on the *contents* would find.

Setting limits. Base each limit on the promise made to users, not on how the pipeline usually behaves. Riverstone's Flash promise is "delivered by 7:00 a.m. IST on working days", so its data should be under 24 hours old at 6:30 a.m., and the limit above (48 hours) is deliberately loose to allow for weekends. Check freshness *before* using a table, and monitor it separately from the pipeline, so a pipeline that never starts is noticed too.

Volume and unusual values

Volume checks compare the size of today's data with what's normal: rows loaded, orders booked, revenue. The classic silent failure is a load that brings in far less than usual, or nothing at all.

Comparing against history needs a baseline. A **median** is a good one, because a single huge day doesn't drag it around the way an average does (Chapter 21):

```

daily = wh.execute("""
    SELECT o.order_date, COUNT(DISTINCT o.order_id) AS orders,
           SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)) AS revenue
    FROM raw.orders AS o JOIN raw.order_items AS i ON i.order_id = o.order_id
    WHERE o.status <> 'Cancelled' AND o.order_date BETWEEN DATE '2025-01-01' AND DATE '2025-12-31'
    GROUP BY o.order_date""").fetchall()
revenues = sorted(r[2] for r in daily)
median = revenues[len(revenues) // 2]
unusual = [(str(d), float(rev)) for d, n, rev in daily if rev > 3 * median]
print("days with orders in 2025:", len(daily))
print("median day's revenue      : Rs", f"{median:,.2f}")
print("days above 3x the median:", len(unusual))
for day, rev in sorted(unusual, key=lambda x: -x[1]):
    print(f" {day} Rs {rev:,.2f}")

```

```

days with orders in 2025: 131
median day's revenue      : Rs 27,442.50
days above 3x the median: 10
2025-11-16 Rs 175,895.00
2025-11-09 Rs 115,847.50
2025-09-13 Rs 108,735.00

```

Reading it. Across 2025, Riverstone had orders on 131 days, and a typical day brought ₹27,442.50. Ten days were more than three times that, the biggest being ₹175,895.00 on 16 November. Those aren't errors: they're large orders and seasonal peaks. That's the lesson of anomaly detection on business data: **unusual is not the same as wrong**. Use these signals to prompt a look, not to block a pipeline.

Sensible volume rules for Riverstone:

- Error if a load that should bring new rows brings **zero** on a working day.
- Error if a published day's revenue is **negative**.
- Warn if a day's revenue is **outside a wide band** around the median of recent days (for example, below a quarter or above five times).
- Warn if row counts in a raw table **fall** compared with the previous load, since sources rarely lose rows.

Watch out: don't learn your baseline from broken days. If the baseline is the average of recent days and last week was broken, "normal" quietly moves toward broken. Exclude days with failed loads from the baseline, or use a longer window and a median.

The three questions to monitor

1. **Did it run?** Pipeline success or failure, and run duration (a run that suddenly takes three times longer is a warning).
2. **Is it fresh?** Age of the newest data in each important table.
3. **Does it look normal?** Row counts and key totals against recent history.

Chapter 46's orchestrator answers the first. This section answers the other two. Together they're what "observability" means for data.

47.6 Lineage: what breaks when this breaks

Lineage is the record of where data comes from and where it goes: which sources feed a table, and which tables, dashboards, and reports depend on it. It answers two questions that come up in every incident:

- **Impact:** this table is wrong. What else is wrong, and who should be told?
- **Cause:** this number is wrong. Which upstream table or source could explain it?

Chapter 46's pipeline already declares its dependencies, so lineage is free. Here's the same idea in a few lines, with two more reports added to show the point:

```
GRAPH = {                                     # asset -> what it is built from (Chapter 46's
pipeline)
    "raw_orders": [], "raw_order_items": [], "raw_dispatch": [], "raw_crm_leads": [],
    "daily_flash": ["raw_orders", "raw_order_items"],
    "deliver_flash": ["daily_flash"],
    "dispatch_report": ["raw_dispatch", "raw_orders"],
    "weekly_pipeline_report": ["raw_crm_leads", "raw_orders"],
}

def downstream_of(asset, graph=GRAPH):
    affected, queue = set(), [asset]
    while queue:
        current = queue.pop()
        for name, parents in graph.items():
            if current in parents and name not in affected:
                affected.add(name); queue.append(name)
    return sorted(affected)

for broken in ["raw_order_items", "raw_dispatch"]:
    print(f"{broken} is broken -> affected: {downstream_of(broken)}")
```

```
raw_order_items is broken -> affected: ['daily_flash', 'deliver_flash']
raw_dispatch is broken -> affected: ['dispatch_report']
```

Reading it. Broken order lines affect the Flash and its delivery. A broken dispatch file affects only the dispatch report. That's the difference between an alert to the sales managers and a quiet note to the warehouse team, and it's why the Flash isn't wired to depend on dispatches.

In a real platform, lineage comes from the orchestrator's graph (Dagster's asset graph, Airflow datasets), from dbt's model graph, and from tools that read query logs to see which dashboards touched which tables. **Column-level lineage** goes further, tracing which source column feeds which report field, which helps when a single measure looks wrong.

Interview extra point. "A dashboard shows wrong numbers. Walk me through it." A strong answer starts with lineage and freshness: which tables feed it, when were they last loaded, did their tests pass, and what changed upstream today. Chapter 77 has more of these.

47.7 Data contracts in practice

Chapter 45 ended with Bhiwandi Main renaming a column and breaking a load. A **data contract** is a written agreement between a data producer and the data team about what will be delivered: the fields, their types and rules, when it arrives, who owns it, and how much notice you get before it changes.

Contracts work best when they're small, written in plain language, and **checked automatically**, so a violation is caught by the pipeline, not by an argument in a meeting.

```
DISPATCH_CONTRACT = {
    "dataset": "bhiwandi_dispatch_sheet",
    "owner": "Bhiwandi Main warehouse supervisor",
    "consumer": "data team (raw.dispatch)",
    "delivered_by": "20:00 IST on the day of dispatch",
    "change_notice_days": 10,
    "columns": {
        "dispatch_id": {"type": "text", "unique": True, "nullable": False},
        "order_id": {"type": "integer", "nullable": False},
        "dispatch_date": {"type": "date dd-mm-yyyy", "nullable": False},
        "warehouse": {"type": "text", "nullable": False},
        "qty_units": {"type": "integer", "nullable": True, "note": "blank allowed while a count is pending"},
        "remarks": {"type": "text", "nullable": True},
    },
}

def check_contract(path, contract):
    found = [r[0] for r in wh.execute(f"DESCRIBE SELECT * FROM read_csv('{path}', all_varchar = true)").fetchall()]
    expected = list(contract["columns"])
    violations = [f"missing column: {c}" for c in expected if c not in found]
    violations += [f"undeclared column: {c}" for c in found if c not in expected]
    return violations

for f in ["exports/dispatch_2026-01-02.csv", "exports/dispatch_2026-01-05.csv"]:
    v = check_contract(f, DISPATCH_CONTRACT)
    print(f"{os.path.basename(f)}: {'meets the contract' if not v else v}")
```

```
dispatch_2026-01-02.csv: meets the contract
dispatch_2026-01-05.csv: ['missing column: qty_units', 'undeclared column: units', 'undeclared column: vehicle_no']
```

Reading it. The January 2 file meets the contract. The January 5 file breaks it in three ways: the agreed `qty_units` column is gone, and two columns nobody agreed to have appeared. The pipeline stops, the owner is told exactly what changed, and nobody has to guess whether `units` means the same thing.

What a good contract contains

Part	Riverstone's dispatch sheet
Dataset and purpose	Daily dispatch sheet; feeds <code>raw.dispatch</code> and the dispatch report
Producer and consumer	Bhiwandi Main warehouse supervisor → data team
Fields	Names, types, whether blanks are allowed, and what each means

Part	Riverstone's dispatch sheet
Rules	dispatch_id unique; order_id exists in the ERP; quantity a whole number
Delivery	One file per working day by 20:00 IST, named <code>dispatch_YYYY-MM-DD.csv</code>
Change process	10 working days' notice; new columns may be added, existing ones not renamed or removed without agreement
What happens on a breach	Load stops; alert to both sides; previous data stays published
Review	Revisited every six months, or when the process changes

Making contracts work with people

- **Write it with the producer, not at them.** The warehouse supervisor isn't trying to break your pipeline; they added a column because a manager asked for vehicle numbers.
- **Explain the cost of a surprise change** in their terms: "when the column changed, the dispatch report was unavailable for a day."
- **Make the safe path the obvious one:** adding columns is always allowed; renaming needs a note. Most breakages come from renames and type changes.
- **Version the contract,** and keep it in Git next to the pipeline code.
- **Contracts go both ways.** When your team publishes data others depend on (Chapter 51), you owe them the same promises.

47.8 Handling a data incident

When a check fails at 6:30 a.m., what happens next should be routine, not improvised.

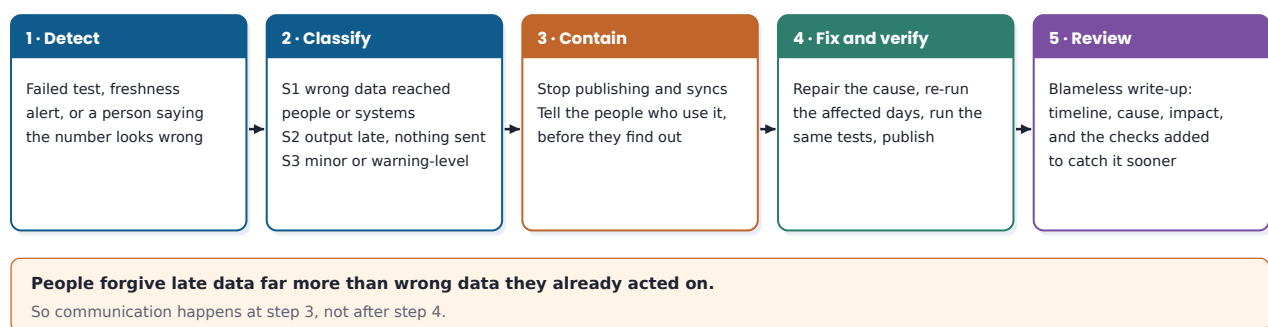


Figure 47.2 — A data incident, step by step. Communication happens before the fix, not after.

1. Detect. From a failed test, a freshness alert, or a person. Treat "a manager says the number looks wrong" as an incident: it means detection failed.

2. Classify. Severity decides how much noise to make.

Severity	Meaning	Riverstone example	Response
S1	Wrong data has reached people or another system, or a critical output is unavailable	Wrong credit limits synced into the ERP; the Flash sent with wrong numbers	Immediate: tell recipients, correct, then fix
S2	A daily output is late or blocked; no wrong numbers went out	5 January's Flash not published	Same morning; tell users the report is late
S3	A non-critical problem or a warning-level test failure	One dispatch row points at an unknown order	Next working day; fix in normal work

3. Contain. Stop publishing and stop side effects: the blocked publish in section 47.4 is containment. Then tell the people who use the data, in their words: *"The Daily Sales Flash for 5 January is delayed. The order lines didn't load, so the numbers would have shown zero sales. Nothing has been sent. We expect it by 11:00."* People forgive late data far more than wrong data they acted on.

4. Fix and verify. Repair the cause, re-run the affected days (Chapter 46's partitions and idempotency), run the same tests, and publish. If wrong numbers did go out, send a labeled correction.

5. Review. A short write-up, within a few days, blameless and factual: what happened, the timeline, the cause, the impact, what was done, and what will stop it or catch it faster next time. For the 5 January incident, the added checks are `orders_have_at_least_one_line` and a volume check for "zero new order lines on a day with new orders".

Real-life example: the missing piece in most incidents. Teams usually fix the data and forget step 5. Six weeks later the same failure happens, and nobody remembers the fix. Ten minutes of writing, kept next to the runbook, is what turns an incident into an improvement.

47.9 Alerts people still read

Alerting fails in two directions: too few, and too many. **Alert fatigue** is what happens when a channel fills with messages nobody acts on, and the real one scrolls past.

Grouping one incident into one message

A single failure usually trips several signals. Sending five messages makes it look like five problems:

```
def group_into_one_alert(day, test_results, freshness_failures, contract_violations):
    failures = [r for r in test_results if r["failing_rows"] and r["severity"] == "error"]
    warnings = [r for r in test_results if r["failing_rows"] and r["severity"] == "warn"]
    if not (failures or freshness_failures or contract_violations):
        return None
    lines = [f"DATA INCIDENT - {day} - Daily Sales Flash not published",
            f"Severity: S2 (a daily report is late; no wrong numbers were sent)", ""]
    if failures:
        lines.append("Failed tests (errors):")
        lines += [f" - {r['name']}: {r['failing_rows']} failing row(s)" for r in failures]
    if warnings:
        lines.append("Warnings (not blocking):")
        lines += [f" - {r['name']}: {r['failing_rows']} row(s)" for r in warnings]
    if freshness_failures:
        lines.append("Stale data: " + ", ".join(freshness_failures))
    if contract_violations:
        lines.append("Contract violations: " + "; ".join(contract_violations))
    lines += [f"", "Impact: sales managers have no Flash for this day; mart.daily_flash is
unchanged.",
            "Next step: fix the order-line load, re-run the day, and publish.",
            "Owner: data team on-call. Runbook: docs/runbooks/daily_flash.md"]
    return "\n".join(lines)

alert = group_into_one_alert("2026-01-05", run_tests(TESTS), ["raw.dispatch (2 days old)",
                                                             check_contract("exports/dispatch_2026-01-05.csv", DISPATCH_CONTRACT)])
print()
print(alert)
```

```
orders_unique_key          PASS    failing rows: 0
orders_customer_not_null   PASS    failing rows: 0
orders_status_accepted     PASS    failing rows: 0
order_items_have_an_order  PASS    failing rows: 0
order_items_quantity_positive PASS    failing rows: 0
orders_have_at_least_one_line ERROR    failing rows: 1  e.g. [(10178,)]
dispatch_order_known       PASS    failing rows: 0

7 tests, 6 passed, 1 failed as errors, 0 warnings

DATA INCIDENT - 2026-01-05 - Daily Sales Flash not published
Severity: S2 (a daily report is late; no wrong numbers were sent)

Failed tests (errors):
- orders_have_at_least_one_line: 1 failing row(s)
Stale data: raw.dispatch (2 days old)
Contract violations: missing column: qty_units; undeclared column: units; undeclared column:
vehicle_no

Impact: sales managers have no Flash for this day; mart.daily_flash is unchanged.
Next step: fix the order-line load, re-run the day, and publish.
Owner: data team on-call. Runbook: docs/runbooks/daily_flash.md
```

Reading it. One message: the failed test, the stale table, the contract violation, the impact, the next step, the owner, and the runbook. In a real deployment this goes to the on-call channel and, for an S1, to the people who received wrong numbers.

Rules that keep alerts useful

- **One incident, one alert.** Group by run and by root cause, as above.

- **Only alert on things someone will do something about.** Everything else goes on a dashboard and is reviewed weekly.
- **Route by owner.** The dispatch contract breach belongs to the warehouse team, not the whole data channel.
- **Say what to do.** Every alert names the next step and links a runbook.
- **Suppress repeats.** While an incident is open, don't re-send the same alert every 15 minutes.
- **Review noisy alerts monthly.** Any alert that fired more than a few times without action is fixed, downgraded, or deleted.
- **Alert on the absence of runs,** not only on failures, or a pipeline that never starts goes unnoticed.

47.10 The tools landscape

Everything in this chapter is SQL, Python, and habits, which is why it works anywhere. Tools add scheduling, history, reporting, and someone else's maintenance.

Tool	What it is	Strengths	Watch for
dbt tests	Tests declared next to your models (47.3)	Free with dbt; runs where your models run; version-controlled; most tests in one line	Only covers data dbt builds; results need somewhere to live
Great Expectations	A Python library of "expectations" with data docs	Large library of checks; profiling; readable reports	Heavier to set up; the API has changed a lot between versions
Soda	Checks in a small declarative language, run by a CLI or library	Readable checks; works across warehouses; hosted option	Another service to run; advanced features are paid
Observability platforms (for example Monte Carlo, Metaplane, Elementary)	Monitor freshness, volume, schema, and lineage automatically	Little to write; anomaly detection and lineage out of the box	Cost; alerts you didn't design; still needs owners
Orchestrator checks (Dagster asset checks, Airflow tasks)	Checks attached to pipeline steps (Chapter 46)	Naturally blocking; visible with the run	Not a full test library on their own
Warehouse constraints	Primary keys, not-null, foreign keys in the database	Stops bad data being written at all	Support varies by warehouse; some are declarative only

A practical combination for a team Riverstone's size: **dbt tests** for the models, **orchestrator checks** for the blocking ones before delivery, **a few freshness and volume monitors**, and **contracts** with the two or three sources that break most often. Add a platform when the number of tables outgrows the team's attention.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Testing only that the pipeline ran	Green runs, wrong numbers	Test the data, not only the job (47.2)
Publishing first and checking after	Wrong rows visible while you investigate	Write-audit-publish (47.4)
Every test an error	Pipelines blocked by trivia; people disable tests	Severity per test; warnings for edges (47.2)
Tests that have never failed	False confidence	Break things on purpose and watch them fail (47.3)
No freshness monitoring	Dashboards quietly show last week	Monitor age of newest data per table (47.5)
Baselines learned from broken days	"Normal" drifts toward broken	Exclude failed days; use medians over longer windows
Treating unusual as wrong	Blocked pipelines on genuine big sales days	Anomalies warn; only hard rules block (47.5)
A missing day that looks like a zero day	Charts show a dip that never happened	Publish load status alongside data (47.4)
No lineage	"Is anything else affected?" takes hours	Declare dependencies; use the orchestrator graph (47.6)
Contracts as documents nobody checks	Same breakage every quarter	Check the contract in the pipeline (47.7)
Blaming the source's owner	Cooperation stops; surprises continue	Make the safe path the obvious one; explain impact in their terms
Fixing without telling users	People act on numbers they didn't know were wrong	Communicate at containment, before the fix (47.8)
Skipping the review	The same incident twice	Ten-minute blameless write-up, kept with the runbook
One alert per failed signal	Five messages for one problem; fatigue	Group by incident; route by owner (47.9)
Alerting on everything	The channel becomes noise	Alert only what someone will act on; review monthly

In the real world: the dashboard that was right but late

Riverstone's sales dashboard had been correct for months, so when Vikram opened it on a Thursday to prepare for a customer call, he didn't check the date on it. He told the customer their last order had been in November.

It had been in January. The dashboard was showing data that stopped on 4 January, three days earlier.

Meera traced it in twenty minutes. On the Monday, a change in the warehouse had renamed a column in the staging model. The nightly pipeline's transformation step had failed at 2:10 a.m. That failure had sent an alert, exactly as designed, into the data team's channel, where it sat among the eleven other

messages the pipeline sent every night: three warnings about dispatch rows with unknown orders, a note about API retries, a long "run finished" message, and a nightly summary nobody read. The failure was message seven.

Meanwhile, every test that ran was green, because the tests all ran against tables that had stopped being updated. Their data was perfectly consistent, and three days old.

The fixes were small, and they're the ones in this chapter.

- **Freshness checks on the tables people read**, not only tests on their contents. The dashboard's own table now fails a check if its newest day is more than 24 hours old on a working day.
- **Fewer alerts, better routed**. Successes stopped being announced. Warnings moved to a weekly digest. Failures went to the on-call person with a subject line naming the table and the impact.
- **A status banner on the dashboard**, driven by the load status table: "Data current to 7 January, 6:35 a.m." Vikram's habit of glancing at the top-right corner now catches in one second what took three days.
- **A contract conversation** with the analytics engineer who had renamed the column, agreeing that renames in shared models need a note and a deprecation window, exactly as for external sources.

At the next sales review, Anita asked whether the data was reliable now. "It fails more often than it used to," Meera said. "That's the point. When it's wrong, it stops and tells us, instead of looking right."

What made this work.

- **They monitored freshness, not only correctness**. The data was correct; that was never the problem.
- **They treated alert noise as a real defect**, because it's what hid the failure.
- **They put the status where the users look**, so trust didn't depend on the data team noticing first.
- **They applied contracts internally**, where most surprise changes actually come from.

Tools

- **Python 3.12** with `duckdb` and `psycopg2`; **PostgreSQL 16**; the **Chapter 47 companion folder** (`companion/ch47/`), which is Chapter 46's environment plus `reset_ch47.py` . Run Python from that folder.
 - **No data quality product needed**: every test in this chapter is SQL run from Python.
 - **Worth knowing about, for later**: dbt tests (Chapter 32), Great Expectations, Soda, Elementary, and the observability platforms in section 47.10.
 - **Versions used for the outputs shown**: Python 3.12.3, DuckDB 1.5.5, psycopg2 2.9.13, PostgreSQL 16.
-

The project: quality, monitoring, and a contract for the Riverstone pipeline

Goal: make the Chapter 46 pipeline trustworthy: nothing wrong gets published, staleness is noticed, one incident produces one useful alert, and the dispatch file has a contract that's checked automatically.

Option A: Riverstone. Extend your Chapter 46 pipeline.

Option B: your own pipeline. Apply the same layers to a report you produce, using data you're allowed to use.

Steps

1. **Write the specification** for two tables: what "good" means, in sentences, for at least four dimensions from section 47.1.
2. **Build a test suite** of at least ten tests across raw, staging, and mart, each with a severity and a one-line description.
3. **Break each rule on purpose** and confirm every test fails when it should. Note any test that didn't catch what you expected.
4. **Convert the pipeline to write-audit-publish** for the Flash, and prove a failing day leaves the published table untouched.
5. **Add freshness checks** for every table people read, with limits based on the promise to users, and a `load_status` table the dashboard can show.
6. **Add two volume checks:** zero rows on a working day (error), and a wide band around the median of recent days (warning).
7. **Add lineage:** list, for each source, which outputs it affects, and use it in your alert text.
8. **Write a data contract** for the dispatch file with the warehouse team's view in mind, and check it in the pipeline.
9. **Run an incident:** break the pipeline, produce one grouped alert, write the user-facing message, fix, re-run, and write the review (half a page).
10. **Count your alerts.** Over a week of runs, how many messages would a person receive? If it's more than a handful, cut them.

Stretch goals

- Store every test result in a `test_results` table with the run and day, and chart pass rates over time.
- Add a check that compares this week's revenue by segment with last week's and warns on large shifts.
- Write the same tests as dbt tests and run both, comparing effort and output.

You've got it when...

- ☐ You can name the six data quality dimensions and write a test for each on Riverstone data.
 - ☐ You can explain why a data test is "a query that returns the rows that break the rule".
 - ☐ You can choose error or warning severity for a test, and defend the choice.
 - ☐ You can say which checks belong in raw, staging, mart, and before delivery.
 - ☐ You can break a rule on purpose and show the test catching it.
 - ☐ You can implement write-audit-publish and prove readers never see failed data.
 - ☐ You can explain why a re-run repairs a table loaded by hash comparison.
 - ☐ You can set freshness limits from a promise to users, and explain why stale data is the hardest failure to notice.
 - ☐ You can build a volume baseline with a median, and explain why unusual is not the same as wrong.
 - ☐ You can use lineage to say who is affected by a broken table.
 - ☐ You can write a data contract and check it automatically.
 - ☐ You can classify an incident's severity, write the user-facing message, and run the review.
 - ☐ You can group one incident into one alert and explain three rules that prevent alert fatigue.
 - ☐ You can recommend a starting set of tools for a small data team.
-

Recap

- Data quality has **dimensions**: accuracy, completeness, validity, uniqueness, consistency, and timeliness. Each becomes a test.
- A **data test** is a query that returns the rows that break a rule; zero rows means it passed. dbt, Great Expectations, and Soda work this way underneath.
- **Severity** decides what stops a pipeline. Warnings are for edges; errors are for numbers people act on. Tests nobody acts on should be fixed, downgraded, or deleted.
- Tests belong at every layer: structure in **raw**, validity and consistency in **staging**, business rules and reconciliation in **mart**, and blocking checks **before delivery**.
- **Break your tests on purpose**. A suite that has never failed hasn't been tested.
- **Write-audit-publish** keeps failed data out of the tables people read; the same 5 January failure that left a wrong row in Chapter 46 now leaves the published table untouched.
- **Observability** answers three questions: did it run, is it **fresh**, does its **volume** look normal. Stale data is the most common and least noticed incident.
- **Unusual is not wrong**. Anomalies warn; only hard rules block.
- **Lineage** tells you what a broken table affects, and where a wrong number could come from.
- **Data contracts** state fields, rules, delivery, change notice, and what happens on a breach, and are checked in the pipeline. They apply to internal producers too.

- **Incidents** follow detect, classify, contain, fix and verify, review. Tell users at containment, before the fix.
 - **Alert fatigue** hides real failures: one incident, one alert; route by owner; alert only what someone will act on.
-

Practice exercises

Run `reset()` and re-run the setup and test blocks before each exercise that uses the companion environment.

Warm-up

1. Name the data quality dimension each rule belongs to: (a) every order line points at an order that exists; (b) `status` is one of four values; (c) yesterday's orders are loaded by 6:30 a.m.; (d) the warehouse's revenue for a day equals the ERP's; (e) one row per order.
2. Write the failing-rows query for: (a) duplicate `dispatch_id` values in `raw.dispatch`; (b) orders with a `sales_rep_id` that doesn't exist in a `raw.employees` table.
3. For each, choose error or warning and give a reason: (a) revenue doesn't match the ERP; (b) a dispatch row points at an unknown order; (c) a customer's email is missing; (d) `mart.daily_flash` has two rows for one day.

Core

4. Add a test `flash_revenue_matches_lines` that fails when a published Flash row's revenue doesn't equal the revenue recomputed from `raw.orders` and `raw.order_items` for that day. Would this have caught the 5 January failure? Would it have caught a wrong `discount` in the source?
5. In section 47.3 the ERP refused the orphan order line, and the warehouse accepted it. Explain why, and say what you'd add to the warehouse to make it refuse too. What's the trade-off of adding it?
6. After the `resync` block, all seven tests pass again. Explain, in terms of Chapter 45's hash comparison, why re-running the load repaired the warehouse. Which of the three broken things would a load with an ID watermark have repaired?
7. The freshness check flagged `mart.daily_flash` as stale at 6:30 a.m. on 6 January. Write the user-facing message the sales team should receive, in three sentences.
8. Set freshness limits for: (a) `raw.orders`; (b) `raw.dispatch`; (c) `raw.crm_leads`, given the Flash is promised by 7:00 a.m. IST on working days, the dispatch file is due at 20:00 IST daily, and lead figures are used in a Monday report. State your assumption about weekends.
9. Using the 2025 numbers in section 47.5 (median ₹27,442.50, 131 days with orders), design two volume rules: one error and one warning. Say what each would have done on 16 November 2025 (₹175,895.00) and on a day the load brought in nothing.

Stretch

- l0. Design the `load_status` table mentioned in section 47.4: columns, one row per what, and how a dashboard would use it to show "data current to...". Include how it distinguishes "not published yet", "published", and "failed".
- l1. Write the full dispatch data contract as a document the warehouse supervisor would accept: fields, rules, delivery, change process, breach handling, review. Then list the three checks you'd automate first, and why those three.
- l2. An analytics engineer renames a column in a shared staging model, breaking two downstream reports. Design an internal contract process for shared models: what's promised, how changes are announced, how long the old name survives, and how the pipeline enforces it.

Think about it (no code needed)

- l3. Riverstone's pipeline now fails more often than it used to, because it stops when data is wrong. Write the paragraph you'd use to explain to Anita why that's an improvement, and what you'd show her each month to prove it.
- l4. Which is worse: a wrong number delivered on time, or no number at all? Argue both sides using examples from this chapter, then say what design choice follows from your answer.

Key terms

data quality dimensions (accuracy, completeness, validity, uniqueness, consistency, timeliness) · data test · failing rows · severity (error, warning) · raw, staging, mart layers · reconciliation · foreign key · self-healing load · dbt test · write-audit-publish (WAP) · audit schema · publish swap · blue/green tables · load status · observability · freshness · staleness limit · volume check · baseline · median · anomaly · lineage · column-level lineage · impact analysis · data contract · change notice · breach · data incident · severity levels (S1, S2, S3) · containment · blameless review · alert fatigue · alert grouping · runbook

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 48, Big Data & Distributed Compute**, scales the pipeline's heavy steps, where tests must run on far more data than fits in memory.
- **Chapter 49, Storage, Warehouses & Lakehouses**, provides the transactions, snapshots, and time travel that make publishing and rollback quick.
- **Chapter 50, Streaming & Real-Time**, asks how to check data that never stops arriving.
- **Chapter 51, Data Activation**, sends warehouse data into the CRM and ERP, where an S1 incident means wrong data in a business system, not only a wrong chart.

- **Chapter 32** covers dbt models, tests, and documentation; **Chapter 14** covers the analyst-level version of this work.
- **Chapter 64, Security, Privacy, Governance & Responsible AI**, covers ownership, stewardship, and the governance that data contracts sit inside.
- **Part VIII:** data quality and incident questions appear in the data engineering interview chapters, and "walk me through a wrong dashboard" is a standard system design question (Chapter 77).

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Consistency (a relationship between tables). (b) Validity. (c) Timeliness. (d) Accuracy. (e) Uniqueness. Completeness would cover "every order has a customer".

2. Sample answers:

```
-- (a) duplicate dispatch_id
SELECT dispatch_id, COUNT(*) AS n
FROM raw.dispatch
GROUP BY dispatch_id
HAVING COUNT(*) > 1;

-- (b) orders whose sales rep is unknown
SELECT o.order_id, o.sales_rep_id
FROM raw.orders AS o
LEFT JOIN raw.employees AS e ON e.employee_id = o.sales_rep_id
WHERE o.sales_rep_id IS NOT NULL AND e.employee_id IS NULL;
```

Both follow the rule: return the rows that break the rule, so zero rows means the test passed. In (b), the `IS NOT NULL` keeps "no rep assigned" out of this test; that's a separate completeness rule.

3. (a) **Error**: the Flash would mislead the sales team. (b) **Warning**: a dispatch may legitimately arrive before its order is loaded, and holding the Flash for it would be worse than publishing. (c) **Warning** in most cases: a missing email doesn't make the sales numbers wrong, though it would be an error for a table that feeds an email campaign. (d) **Error**: duplicate rows double revenue, and it signals a non-idempotent step.

4. A sample test:

```
data_test("flash_revenue_matches_lines", ""
SELECT f.flash_date, f.revenue_booked, COALESCE(c.revenue, 0) AS recomputed
FROM mart.daily_flash AS f
LEFT JOIN (
    SELECT o.order_date, SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)) AS revenue
    FROM raw.orders AS o JOIN raw.order_items AS i ON i.order_id = o.order_id
    WHERE o.status <> 'Cancelled' GROUP BY o.order_date) AS c
ON c.order_date = f.flash_date
WHERE f.revenue_booked <> COALESCE(c.revenue, 0)""")
```

It would **not** have caught the 5 January failure on its own: the Flash was built from those same warehouse tables, so both sides would have agreed on ₹0. That's why the reconciliation test compares with the **ERP**, not with the warehouse. It also would **not** catch a wrong discount in the source, for the same reason: the warehouse faithfully copies the source's mistake. It's still useful, because it catches a wrong transformation, publishing the wrong day, or a stale Flash row.

5. The ERP has a **foreign key** on `order_items.order_id`, so the database itself refuses a line whose order doesn't exist. The warehouse table has no such constraint, so it accepted the row. You could add one in DuckDB or your warehouse where supported. The trade-offs: constraints are checked on every write, which slows large loads; they force load *order* (orders before lines) and make partial loads fail rather than land; and many cloud warehouses don't enforce them at all, which is why data tests exist. A common compromise is a primary key on raw tables (Chapter 45's idempotency needs it) plus relationship tests instead of enforced foreign keys.

6. The hash comparison builds the full picture of what the source contains and makes the warehouse match it: rows in the warehouse but not the source are **deleted**, and rows whose values differ are **updated**. The two invented order lines existed nowhere in the source, so they were deleted; order 10177's status differed, so it was updated back to the source's value. An **ID watermark** load would have repaired **none of them**: it only inserts rows with IDs above the watermark, so it never deletes and never updates. That's Chapter 45's lesson again, now visible as a data quality property.

7. A sample message: *"The Daily Sales Flash for 5 January hasn't been published. Yesterday's order lines didn't load, so today's figures would have shown no sales, which would be wrong. We're fixing the load now and expect the Flash by 11:00; the dashboard still shows data up to 2 January."* Three things make it good: no jargon, the reason the report was held, and when to expect it.

8. Assuming Riverstone doesn't dispatch or book orders on Sundays: (a) `raw.orders` : **26 hours** on working days (yesterday's data must be there by 6:30, with a little slack), with the check skipped or widened after a non-working day. (b) `raw.dispatch` : **12 hours** at 8:00 a.m. (due at 20:00 the previous evening), again only on days following a working day. (c) `raw.crm_leads` : **24 hours** for the Monday report, so a warning on other days and an error if it's more than 3 days old on Monday morning. Any answer is acceptable if the limit comes from a promise and the weekend assumption is stated.

9. **Error rule**: on a working day, the day's booked revenue is **zero** while the ERP has orders for that day, or the published revenue is negative. On 16 November this does nothing (₹175,895.00 is not zero); on the day the load brought nothing, it fires and blocks publishing. **Warning rule**: the day's revenue is below **a quarter** of the recent median (₹6,860) or above **five times** it (₹137,212). On 16 November this warns, and a human confirms that a genuinely large order arrived; on the empty-load day the error rule has already blocked it. Using three times the median as an *error* would have blocked ten legitimate days in 2025, which is exactly the mistake section 47.5 warns against.

10. A sample design: one row per **table and day** (or per published partition): `table_name`, `business_date`, `status` (`pending`, `published`, `failed`, `late`), `run_id`, `published_at`, `rows`, `failed_tests`, `note`. The pipeline inserts `pending` when a run starts, updates to `published` after a successful publish, and `failed` when the audit blocks it; a separate freshness monitor marks a `pending` row `late` when it passes its deadline. A dashboard reads the latest `published` row for its

table to show "Data current to 7 January, 6:35 a.m.", and shows an amber banner when today's status is `late` or `failed`. The important part is that "we have no data for today" is a *recorded state*, not an absence, so charts can show it instead of drawing a zero.

11. A good answer covers: **dataset and purpose** (daily dispatch sheet, feeds the dispatch report used by customer service); **fields** with types, meanings, and whether blanks are allowed, including that `qty_units` may be blank while a count is pending; **rules** (`dispatch_id` unique, `order_id` exists in the ERP, quantities whole numbers, date as dd-mm-yyyy); **delivery** (one file per working day by 20:00 IST, named `dispatch_YYYY-MM-DD.csv`, in the shared folder); **change process** (new columns allowed any time; renames, removals, and type changes need 10 working days' notice and a quick call); **breach handling** (load stops, both sides alerted, previous data stays published, the day is loaded once corrected); **review** every six months; **contacts** on both sides. The three checks to automate first: (1) **columns match the contract**, because renames caused the last two breakages; (2) **file arrived by the deadline**, because a missing file is the most common and least visible failure; (3) **dispatch_id unique**, because duplicates silently double dispatched quantities. Each is cheap and catches a failure that has already happened.

12. A workable internal process: shared models are **owned** by a named person, and their column names are treated as an interface. Changes are announced in the data channel and in the model's documentation, with the reason. A rename is done as **add, then deprecate**: the new column is added alongside the old one, the old one is marked deprecated with a removal date at least two weeks out, consumers are told which reports use it (from lineage), and only then is it removed. Enforcement lives in the pipeline: a test asserts that every column listed in the model's contract file still exists, so a removal without a contract change fails in the pull request, not at 6:30 a.m. Chapter 32's dbt contracts and Chapter 29's code review make this routine rather than personal.

13. A sample paragraph: *"The pipeline stops more often because we added checks that compare our numbers with the ERP before anything is published. When it stops, the old numbers stay up and you get a message telling you what's late and when to expect it. Before, the same problems still happened; we published them, and someone found out in a meeting. What I'd like to show you each month: how many times we stopped publication and why, how many wrong numbers reached anyone (the number I want at zero), and how long it took us to publish after a problem. If the first number is high and the second is zero, the system is working."* The key move is to reframe visible failures as caught failures, with metrics that separate the two.

14. A *wrong number on time* is worse in most business settings: people act on it, decisions are made, and the error spreads into other systems and into trust, as the Chapter 46 story of duplicate emails showed. No *number* is worse when a decision must be made at a fixed time and a rough figure would be better than nothing, for example a dispatch cut-off, or where the absence itself misleads (a chart showing zero sales). The design that follows from the first view is this chapter's: block publication, keep the last good data, tell people, and fix. The design that follows from the second is to publish with an explicit, visible warning about what's missing or provisional, so people can decide for themselves. A good answer picks one and names the conditions where it would switch.